IoT-BASED PREDICTIVE MAINTENANCE AND CONDITION MONITORING OF AN INDUSTRIAL INDUCTION MOTOR USING ESP32 AND MQTT

A PROJECT REPORT

Submitted by

SAM BLESSON S

(Roll No. 2023111073)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

MECHANICAL ENGINEERING



COLLEGE OF ENGINEERING GUINDY

ANNA UNIVERSITY : CHENNAI 600 025

OCTOBER 2026

MARKS (30)		
CO3 (10)	**CO4 (10)**	**CO5 (10)**

CO3: Analyze the Communication Protocols and Control Strategies Required for the Implementation of IoT-Based Mechatronic Systems

CO4: Develop a Comprehensive Mechatronics and IoT Toolbox for Effective Integration of Sensors, Actuators, and Communication Systems for Real-World Applications

CO5: Collaborate to Design a Mechatronics and IoT Project Addressing Global Challenges

ABSTRACT

Unexpected failure of industrial induction motors causes production downtime, higher maintenance cost and equipment damage. Periodic inspection alone may not reveal faults while they are developing. This project presents an IoT-based condition monitoring and predictive maintenance prototype for an induction motor, built on an ESP32 microcontroller, the MQTT protocol and a Node-RED Dashboard 2.0 interface. Motor temperature, vibration and current are monitored; in the Wokwi simulation, three potentiometers stand in for the sensors.

The ESP32 converts the analog readings into engineering units, compares them with warning and trip thresholds, calculates a rule-based risk score, classifies the motor condition and publishes JSON telemetry and alarm messages to the HiveMQ public broker. It also accepts start, stop and reset commands. LEDs and a buzzer indicate the running, trip and contactor states. Node-RED subscribes to the telemetry topic and shows the readings on four gauges and a live JSON panel. The risk score is a transparent threshold-based heuristic, not a trained machine-learning model, and the system was evaluated only in simulation.

Keywords: Mechatronics, Internet of Things, ESP32, MQTT, induction motor, predictive maintenance, Node-RED, condition monitoring.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vi
	LIST OF FIGURES	vii
	LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE	viii
1.	INTRODUCTION	1
	1.1 GENERAL	1
	1.2 PROBLEM STATEMENT	1
	1.3 OBJECTIVES	1
	1.4 SCOPE AND ORGANISATION OF THE REPORT	2
2.	BACKGROUND AND RELATED CONCEPTS	3
	2.1 MAINTENANCE STRATEGIES	3
	2.2 MOTOR CONDITION PARAMETERS	3
	2.3 IoT AND THE MQTT PROTOCOL	4
	2.4 NODE-RED AND DASHBOARD 2.0	4
	2.5 WOKWI SIMULATION	4
3.	SYSTEM ARCHITECTURE AND HARDWARE DESIGN	5
	3.1 OVERALL ARCHITECTURE	5
	3.2 HARDWARE COMPONENTS	6
	3.3 CIRCUIT DESIGN AND PIN CONFIGURATION	6
	3.4 SOFTWARE AND PLATFORMS	8
4.	FIRMWARE DESIGN AND METHODOLOGY	9
	4.1 INITIALISATION	9
	4.2 SENSOR DATA ACQUISITION	9
	4.3 THRESHOLD-BASED MONITORING	9
	4.4 RISK SCORE CALCULATION	9

4.5 CONDITION CLASSIFICATION....10
4.6 OUTPUT CONTROL....10
4.7 OPERATING ALGORITHM....11
5. MQTT COMMUNICATION DESIGN.... 13
5.1 PUBLISH-SUBSCRIBE ARRANGEMENT....13
5.2 MQTT TOPICS....13
5.3 TELEMETRY MESSAGE FORMAT.... 13
5.4 ALARM MESSAGES.... 14
5.5 REMOTE COMMANDS.... 14
5.6 RELIABILITY AND SECURITY.... 15
6. NODE-RED DASHBOARD DESIGN.... 16
6.1 FLOW DESIGN....16
6.2 PARSE TELEMETRY FUNCTION....16
6.3 DASHBOARD INTERFACE....16
6.4 DATA PROCESSING FLOW.... 17
7. RESULTS AND DISCUSSION....19
7.1 FUNCTIONAL BEHAVIOUR.... 19
7.2 DISCUSSION.... 19
7.3 VALIDITY OF THE RESULTS....20
8. APPLICATIONS, ADVANTAGES AND LIMITATIONS.....21
8.1 APPLICATIONS.... 21
8.2 ADVANTAGES.... 21
8.3 LIMITATIONS....21
9. COURSE OUTCOME MAPPING....23
9.1 GENERAL.... 23
9.2 CO3: COMMUNICATION PROTOCOLS AND CONTROL STRATEGIES....23
9.3 CO4: MECHATRONICS AND IoT TOOLBOX.... 23
9.4 CO5: ADDRESSING INDUSTRIAL CHALLENGES.....24

10. CONCLUSION AND FUTURE SCOPE.....25
10.1 CONCLUSION.....25
10.2 FUTURE SCOPE.....25
APPENDIX 1 ESP32 FIRMWARE SOURCE CODE.....27
APPENDIX 2 NODE-RED PARSE TELEMETRY FUNCTION AND FLOW CONFIGURATION.....32
REFERENCES.....33

LIST OF TABLES

TABLE NO. **PAGE NO.**

Table 2.1 Comparison of maintenance strategies..... 3

Table 2.2 Monitored parameters and typical indications.....3

Table 3.1 Hardware components and their roles.....6

Table 3.2 Pin configuration of the ESP32.....7

Table 3.3 Simulated measurement ranges.....7

Table 4.1 Warning and trip thresholds..... 9

Table 4.2 Risk score contributions..... 10

Table 4.3 Condition classification logic.....10

Table 4.4 Output states of the ESP32.....10

Table 5.1 MQTT topics used in the project..... 13

Table 5.2 Fields of the telemetry message..... 14

Table 5.3 Remote commands accepted by the ESP32.....14

Table 6.1 Dashboard gauge configuration..... 17

Table 7.1 Condition and risk score produced by the firmware logic.....19

Table 9.1 Mapping of the project to the course outcomes.....23

LIST OF FIGURES

FIGURE NO. **PAGE NO.**

Figure 3.1 Block diagram of the proposed system.....5

Figure 3.2 Wokwi circuit of the motor monitoring simulation.....7

Figure 4.1 Flowchart of the firmware operation..... 11

Figure 5.1 MQTT publish-subscribe paths in the project..... 13

Figure 6.1 Node-RED flow for the motor health dashboard..... 16

Figure 6.2 Node-RED Dashboard 2.0 motor health page (sample display).....17

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

ADC	Analog-to-Digital Converter
A	Ampere (unit of current)
°C	Degree Celsius
CO	Course Outcome
ESP32	Espressif 32-bit Wi-Fi and Bluetooth microcontroller
GPIO	General Purpose Input/Output
HiveMQ	MQTT broker platform (public broker used for demonstration)
IoT	Internet of Things
JSON	JavaScript Object Notation
LED	Light Emitting Diode
mm/s	Millimetre per second (unit of vibration velocity)
MQTT	Message Queuing Telemetry Transport
PLC	Programmable Logic Controller
QoS	Quality of Service (MQTT delivery level)
RMS	Root Mean Square
TLS	Transport Layer Security
UI	User Interface
V	Volt
Wi-Fi	Wireless local area network (IEEE 802.11)

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Mechatronics integrates mechanical systems, electronics, control engineering and computer-based intelligence to build automated products and processes. The Internet of Things (IoT) extends this integration by allowing devices to collect, process and exchange data over communication networks, so that machines can be observed and supervised from a distance.

Induction motors are among the most widely used machines in manufacturing plants, pumping stations, conveyors and building services. Their operating condition changes with excessive mechanical load, bearing deterioration, misalignment, inadequate cooling and electrical abnormalities. Monitoring temperature, vibration and current can help reveal such abnormal operation before it develops into a severe failure.

This project demonstrates the basic architecture of an IoT-enabled condition monitoring system for an induction motor. An ESP32 collects simulated sensor data, evaluates the motor condition and sends the information through MQTT to a Node-RED dashboard, where an operator can observe the parameters and identify warning or critical conditions.

1.2 PROBLEM STATEMENT

Unexpected motor failures interrupt production, increase operating cost and create maintenance difficulties. Manual inspection and fixed-interval maintenance do not give continuous visibility into the changing condition of a motor.

A system is therefore required that continuously monitors important motor parameters, identifies abnormal conditions using predefined thresholds, sends the measurements to a central dashboard and provides a suitable maintenance recommendation. This project addresses the problem with a simulated IoT-based condition monitoring system that combines sensor inputs, embedded control, MQTT communication, risk assessment, visual indicators and a web-based dashboard.

1.3 OBJECTIVES

The objectives of the project are:

1. To develop an ESP32-based system for monitoring motor temperature, vibration and current.
2. To simulate the sensor outputs using analog potentiometers in the Wokwi environment.
3. To implement MQTT communication for transmitting telemetry and receiving remote commands.
4. To develop a rule-based risk assessment algorithm for identifying abnormal operating conditions.
5. To classify the motor condition as normal, warning, high risk or critical.
6. To represent motor operation and critical faults using LEDs and a buzzer.
7. To develop a Node-RED Dashboard 2.0 interface that displays the sensor readings and telemetry data.
8. To demonstrate the integration of sensors, actuators, communication protocols and supervisory monitoring in a mechatronic application.

1.4 SCOPE AND ORGANISATION OF THE REPORT

The work is a simulation-based prototype. No physical induction motor, calibrated sensor or industrial network is used, and the results therefore show the feasibility of the architecture and the monitoring logic rather than the diagnostic accuracy of the system on real equipment.

Chapter 2 reviews the background concepts. Chapter 3 describes the system architecture and hardware design. Chapter 4 explains the firmware and the condition assessment method. Chapter 5 presents the MQTT communication design, and Chapter 6 describes the Node-RED dashboard. Chapter 7 discusses the results. Chapter 8 lists the applications, advantages and limitations, Chapter 9 maps the work to the course outcomes, and Chapter 10 gives the conclusion and future scope.

CHAPTER 2

BACKGROUND AND RELATED CONCEPTS

2.1 MAINTENANCE STRATEGIES

Reactive maintenance responds after the equipment has failed. Preventive maintenance relies on predetermined inspection or service intervals. Condition-based maintenance uses actual equipment measurements to decide when maintenance is needed. Predictive maintenance goes further and estimates future failures from historical data, statistical methods or machine-learning models (Mobley, 2002). Table 2.1 compares these strategies.

Table 2.1 Comparison of maintenance strategies

Strategy	Basis for action	Limitation
Reactive	Maintenance after failure	Unplanned downtime and possible secondary damage
Preventive	Fixed time or usage interval	May service healthy equipment or miss an early fault
Condition-based	Measured condition against limits	Needs sensors and a monitoring system
Predictive	Trend or model-based estimate of future failure	Needs representative historical data and validation

The prototype in this report belongs to the condition-based category: it compares live measurements with fixed limits and advises the operator. It does not estimate remaining useful life.

2.2 MOTOR CONDITION PARAMETERS

Three parameters are monitored in this project. Temperature rises when cooling is poor, when the load is high or when bearings or windings are in distress. Vibration velocity in mm/s RMS is a common indicator of mechanical condition, because bearing wear, misalignment, imbalance

and loose mounting increase it. Motor current reflects the mechanical load and electrical supply condition, and an unusually high current can indicate overload or a developing fault. Table 2.2 summarises these indications.

Table 2.2 Monitored parameters and typical indications

Parameter	Typical indication when abnormal
Temperature	Inadequate cooling, overload, bearing or winding distress
Vibration	Bearing wear, shaft misalignment, imbalance, loose mounting
Current	Mechanical overload, supply abnormality, driven-equipment problem

2.3 IoT AND THE MQTT PROTOCOL

MQTT is a lightweight publish-subscribe messaging protocol widely used in IoT systems (OASIS, 2014). Devices do not connect to each other directly. Publishers send messages on named topics to a broker, and the broker forwards each message to every client that has subscribed to that topic. MQTT defines three levels of Quality of Service: 0 (at most once), 1 (at least once) and 2 (exactly once). The prototype uses QoS 0, which suits low-rate monitoring data in a demonstration.

2.4 NODE-RED AND DASHBOARD 2.0

Node-RED is a flow-based programming tool in which nodes are wired together to receive, process and output data (OpenJS Foundation, n.d.). Dashboard 2.0 adds user-interface nodes such as gauges and templates that are served as a web page in a browser (FlowFuse, n.d.). In this project an MQTT input node receives the telemetry, a function node parses it, and Dashboard nodes display it.

2.5 WOKWI SIMULATION

Wokwi is an online simulator for microcontroller boards and electronic components (Wokwi, n.d.). It runs the ESP32 firmware with virtual components and provides a guest Wi-Fi network, so that networking, MQTT and the control logic can be tested without physical hardware.

CHAPTER 3

SYSTEM ARCHITECTURE AND HARDWARE DESIGN

3.1 OVERALL ARCHITECTURE

The system has five functional blocks, shown in Figure 3.1: simulated input sensors, the ESP32 microcontroller, Wi-Fi and MQTT communication, the MQTT broker with the Node-RED dashboard, and the ESP32 output devices.

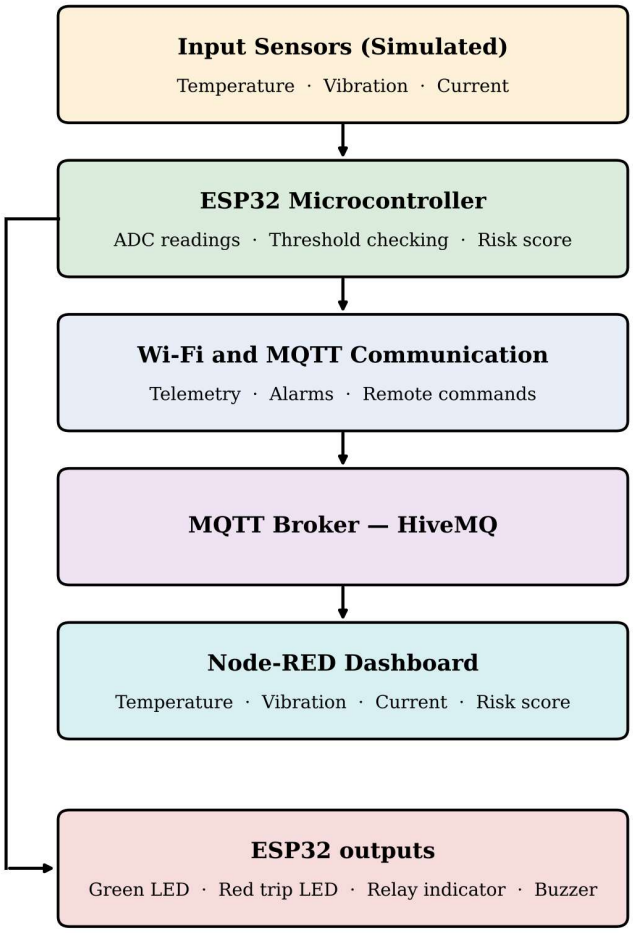


Figure 3.1 Block diagram of the proposed system

The simulated sensors supply analog values for temperature, vibration and current. The ESP32 reads and scales these values, checks them against thresholds and calculates a risk score. It then publishes telemetry, alarms and status over Wi-Fi to the HiveMQ broker, and Node-RED

subscribes to the telemetry topic to drive the dashboard. In parallel, the ESP32 drives the green LED, red trip LED, relay indicator and buzzer to show the motor state locally.

3.1.1 Sensor simulation layer

Three potentiometers are connected to analog input pins of the ESP32. Each represents one motor sensor: temperature, vibration velocity or motor current. Their positions are converted into engineering-unit values by a linear scaling function. The values are simulated and are not measurements of a physical motor.

3.1.2 Embedded processing layer

The ESP32 reads the analog inputs, converts them to engineering units, compares them with warning and trip thresholds, calculates the risk score, classifies the condition and decides the state of every output.

3.1.3 Communication, monitoring and indication layers

The ESP32 joins the Wi-Fi network, publishes JSON messages to the broker and subscribes to a command topic. Node-RED receives the telemetry and presents it on Dashboard 2.0. The output layer consists of the green LED (motor running), red LED (trip), blue LED (simulated contactor) and a buzzer (audible trip alarm).

3.2 HARDWARE COMPONENTS

Table 3.1 Hardware components and their roles

Component	Role in the system
ESP32 development board	Central controller: reads analog inputs, evaluates faults, manages Wi-Fi and MQTT
Three potentiometers	Simulate the temperature, vibration and current sensor outputs
Green LED	Indicates that the simulated motor is running
Red LED	Indicates a critical condition or trip

Component	Role in the system
Blue LED	Represents the simulated contactor (relay) output
Buzzer	Gives an audible indication of a critical trip
Connecting wires	Virtual power and signal connections between the components

3.3 CIRCUIT DESIGN AND PIN CONFIGURATION

The circuit built in Wokwi is shown in Figure 3.2. The pin assignments are listed in Table 3.2. The three sensor inputs use GPIO34, GPIO35 and GPIO32, which are ADC1 channels of the ESP32 (Espressif Systems, n.d.). ADC1 remains usable while Wi-Fi is active, which suits a design that reads sensors and transmits data at the same time.

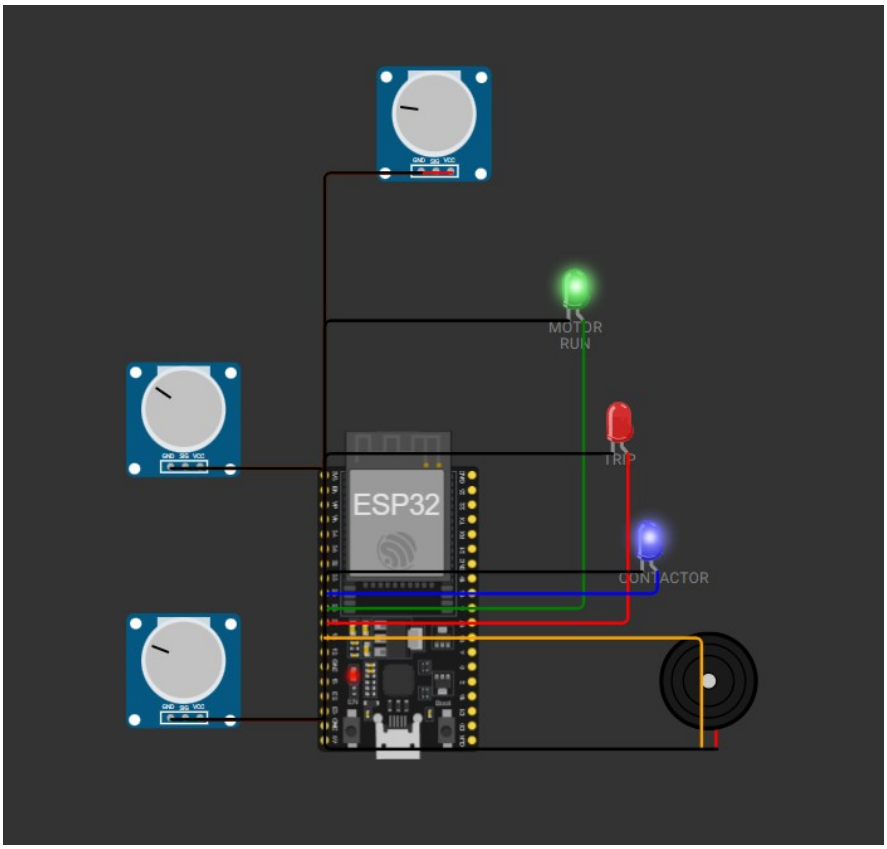


Figure 3.2 Wokwi circuit of the motor monitoring simulation

Table 3.2 Pin configuration of the ESP32

Component	ESP32 GPIO	Function

Component	ESP32 GPIO	Function
Temperature potentiometer	GPIO34	Analog temperature input
Vibration potentiometer	GPIO35	Analog vibration input
Current potentiometer	GPIO32	Analog current input
Green LED	GPIO26	Motor-running indication
Red LED	GPIO27	Critical fault indication
Blue LED	GPIO25	Simulated contactor indication
Buzzer	GPIO14	Critical fault alarm

The ADC is configured for 12-bit resolution, so a reading lies between 0 and 4095. Each reading is scaled linearly into the range of the parameter it represents, as given in Table 3.3 and Equation 3.1.

Table 3.3 Simulated measurement ranges

Parameter	Simulated range	Unit
Temperature	25 to 110	°C
Vibration	0 to 12	mm/s RMS
Motor current	0 to 25	A

$$Scaled\ value = Min + (Max - Min) \times ADC\ reading / 4095 \quad (3.1)$$

This conversion lets each potentiometer position represent a measurement in engineering units.

3.4 SOFTWARE AND PLATFORMS

- **Wokwi:** virtual hardware environment for the ESP32 and the connected components.
- **Arduino framework:** programming and peripheral control for the ESP32.
- **PubSubClient library:** MQTT publishing, subscription and message handling (O'Leary, n.d.).

- **ArduinoJson library:** creation of structured JSON telemetry and alarm messages (Blanchon, n.d.).
- **Node-RED and Dashboard 2.0:** message processing and browser-based visualisation.

CHAPTER 4

FIRMWARE DESIGN AND METHODOLOGY

4.1 INITIALISATION

At start-up the ESP32 initialises serial communication, sets the ADC resolution to 12 bits, configures the output pins and switches all motor indicators off. It then connects to the Wokwi-GUEST Wi-Fi network. When Wi-Fi is connected, it sets the MQTT broker address, registers the callback that handles incoming commands, connects to the broker and subscribes to the command topic.

4.2 SENSOR DATA ACQUISITION

The firmware reads the three analog inputs with analogRead() and scales them using Equation 3.1. A new sample is taken and a telemetry message is published approximately every two seconds, which is adequate for demonstrating low-frequency condition monitoring.

4.3 THRESHOLD-BASED MONITORING

Each measurement is compared with a warning threshold and a trip threshold, given in Table 4.1. A critical trip occurs when any trip threshold is exceeded or the manual trip flag is set.

Table 4.1 Warning and trip thresholds

Parameter	Warning threshold	Trip threshold
Temperature	70 °C	90 °C
Vibration	4.5 mm/s RMS	8 mm/s RMS
Current	12 A	18 A

These are demonstration settings. For a real motor, thresholds must be derived from the motor specification, operating conditions, sensor placement and applicable engineering standards.

4.4 RISK SCORE CALCULATION

The risk score is calculated from the warning status of each parameter using the weights in Table 4.2 and Equation 4.1. A persistence bonus of 10 points is added when at least one warning has been active for three or more consecutive samples, and the score is limited to 100.

Table 4.2 Risk score contributions

Condition	Points
Temperature warning active	25
Vibration warning active	40
Current warning active	25
Persistence: three or more consecutive samples with a warning	10

$$\textit{Risk score} = \textit{Temperature points} + \textit{Vibration points} + \textit{Current points} + \textit{Persistence bonus} \quad (4.1)$$

The score is transparent and easy to interpret, but it is a heuristic indicator. It is not a statistically validated probability of failure and does not estimate remaining useful life.

4.5 CONDITION CLASSIFICATION

The firmware assigns one of four conditions, checked in the order given in Table 4.3. The critical condition has priority over the others.

Table 4.3 Condition classification logic

Condition	Rule	Recommendation issued
CRITICAL	Any trip threshold exceeded, or manual trip active	Inspect the indicated fault before restart

Condition	Rule	Recommendation issued
HIGH RISK	Two or more warnings, or risk score of 60 or more	Schedule inspection; check bearing, cooling and load
WARNING	Exactly one warning	Parameter-specific advice (cooling, bearing or load)
NORMAL	No warning and no trip	Continue operation; routine inspection

4.6 OUTPUT CONTROL

The simulated motor runs only when it is enabled and no trip is active. When a trip occurs, the green LED and the contactor indicator switch off, and the red LED and buzzer switch on. Table 4.4 lists the output states.

Table 4.4 Output states of the ESP32

State	Green LED	Red LED	Contactor LED	Buzzer
Running (enabled, no trip)	ON	OFF	ON	OFF
Stopped by command	OFF	OFF	OFF	OFF
Tripped	OFF	ON	OFF	ON

The start command enables the motor and clears the manual trip flag in this demonstration, the stop command disables the motor, and the reset command clears the manual trip flag. These are simulation-level functions. A real installation needs an independently engineered safety circuit and a verified restart procedure.

4.7 OPERATING ALGORITHM

The operating cycle of the firmware is summarised in Figure 4.1 and in the following steps.

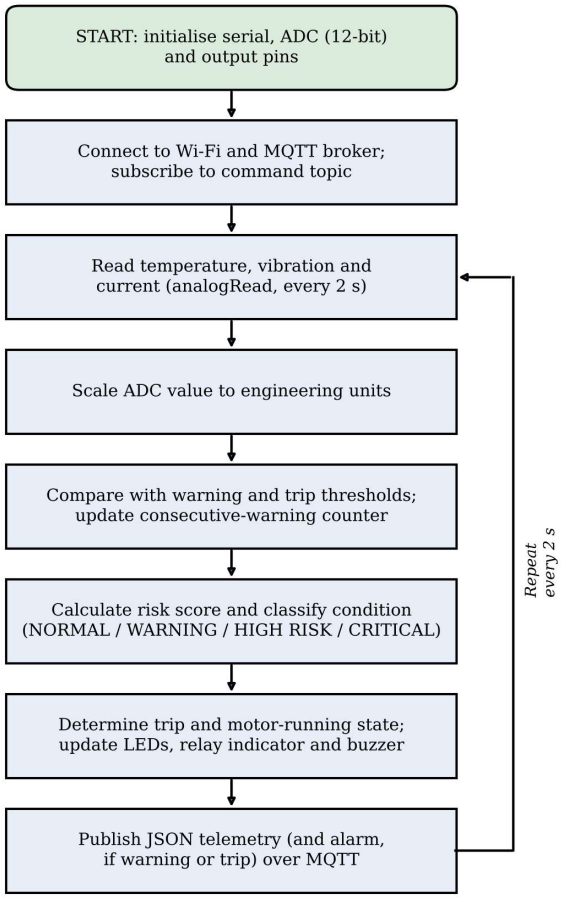


Figure 4.1 Flowchart of the firmware operation

1. Start the ESP32 and initialise serial communication, ADC settings and output pins.
2. Connect to Wi-Fi, connect to the MQTT broker and subscribe to the command topic.
3. Read the temperature, vibration and current inputs.
4. Convert the ADC values into simulated engineering measurements.
5. Compare the measurements with the warning and trip thresholds and update the consecutive-warning counter.
6. Calculate the risk score, classify the condition and generate the recommendation.
7. Determine the motor-running and trip states and update the LEDs and buzzer.

8. Construct the JSON telemetry message and publish it; publish an alarm message when a warning or trip is present.
9. Repeat the cycle.

CHAPTER 5

MQTT COMMUNICATION DESIGN

5.1 PUBLISH-SUBSCRIBE ARRANGEMENT

The ESP32 is both a publisher and a subscriber. It publishes telemetry and alarm messages and subscribes to a command topic. Node-RED subscribes to the telemetry topic. Figure 5.1 shows the message paths through the public HiveMQ broker, which is used for demonstration only.

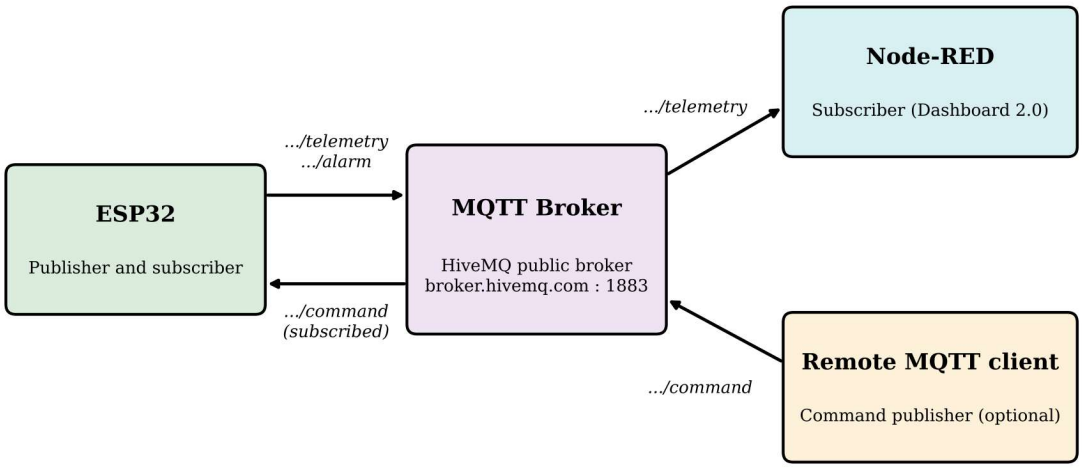


Figure 5.1 MQTT publish-subscribe paths in the project

5.2 MQTT TOPICS

Table 5.1 MQTT topics used in the project

Topic	Purpose
mechatronics/motorpm/demo2026/device01/telemetry	Periodic motor measurements and status
mechatronics/motorpm/demo2026/device01/alarm	Warning and critical-condition messages
mechatronics/motorpm/demo2026/device01/command	Remote start, stop and reset commands

Because the broker is public and shared, the topic suffix should be made unique before several demonstrations are run at the same time.

5.3 TELEMETRY MESSAGE FORMAT

The ESP32 builds a JSON document with the fields in Table 5.2 and publishes it every two seconds. An example is shown below. The values are illustrative and are not a recorded test result.

Table 5.2 Fields of the telemetry message

Field	Description
device	Device identifier (motor-01)
sample	Sample counter
temperature_c	Temperature in °C
vibration_mm_s	Vibration velocity in mm/s RMS
current_a	Motor current in A
risk_score	Rule-based risk score (0 to 100)
condition	NORMAL, WARNING, HIGH RISK or CRITICAL
motor_running	True when the motor is enabled and not tripped
motor_enabled	State of the start and stop command
trip	True when a trip condition is active
recommendation	Maintenance recommendation text
uptime_s	Time since start-up in seconds

```
{
  "device": "motor-01",
  "sample": 25,
  "temperature_c": 75.2,
  "vibration_mm_s": 5.1,
  "current_a": 13.4,
  "risk_score": 90,
  "condition": "HIGH RISK",
  "motor_running": true,
  "motor_enabled": true,
  "trip": false,
  "recommendation": "Schedule inspection soon; check
bearing, cooling and load.",
  "uptime_s": 50
}
```

5.4 ALARM MESSAGES

An alarm message is published on the alarm topic whenever at least one warning is active or a trip occurs. It carries the device identifier, condition, the three measurements and the recommendation. This separates routine telemetry from events that need attention.

5.5 REMOTE COMMANDS

Table 5.3 Remote commands accepted by the ESP32

Command	Effect
start	Enables the motor and clears the manual trip flag
stop	Disables the motor
reset	Clears the manual trip flag

The command text is trimmed and converted to lower case before it is compared, so START and start have the same effect.

5.6 RELIABILITY AND SECURITY

The firmware checks the Wi-Fi connection and retries the MQTT connection every three seconds when it is lost. It services incoming messages with the client loop. The connection uses a public broker on port 1883 without TLS and without authentication, and anyone who knows the topic could send a command. This is acceptable only for a demonstration. A production system needs a private broker with TLS, authenticated clients with unique credentials and topic-level access control.

CHAPTER 6

NODE-RED DASHBOARD DESIGN

6.1 FLOW DESIGN

The Node-RED flow is shown in Figure 6.1. An MQTT input node named Motor telemetry subscribes to the telemetry topic on the HiveMQ broker and shows a connected status. Its output goes to a function node, Parse telemetry, which has five outputs: temperature, vibration, current and risk score go to four gauge nodes, and the complete data object goes to a template node that shows the live telemetry JSON. A debug node attached to the temperature gauge output helps to inspect the value in the Node-RED sidebar.

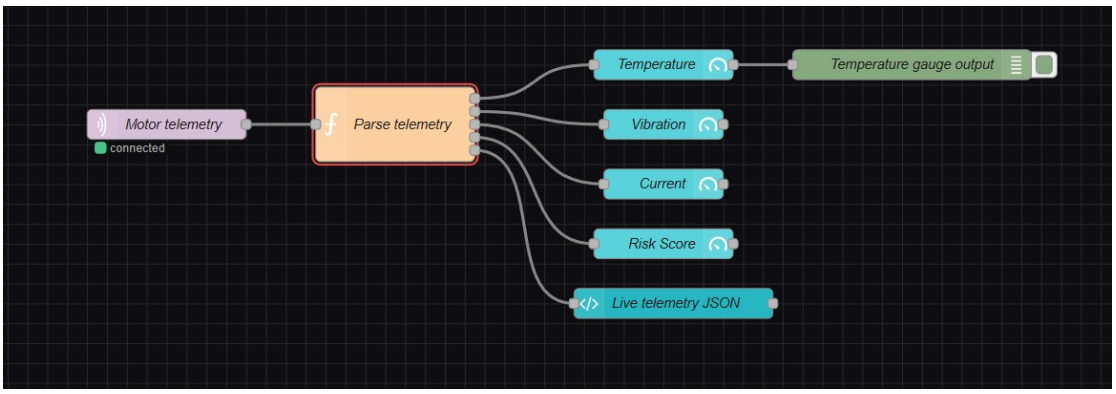


Figure 6.1 Node-RED flow for the motor health dashboard

6.2 PARSE TELEMETRY FUNCTION

The function node accepts the payload as an object or a JSON string, discards messages that cannot be parsed, converts each numeric field to a number and builds an object containing the temperature, vibration, current, risk score, condition, motor state, recommendation and the time of arrival. It returns the four numbers and the object on its five outputs. The code is listed in Appendix 2.

6.3 DASHBOARD INTERFACE

The Dashboard 2.0 page, Motor Health, is shown in Figure 6.2. It has four half-circle gauges with needles and a dark panel that displays the latest parsed telemetry as formatted JSON. The gauge ranges and colour bands are given in Table 6.1.

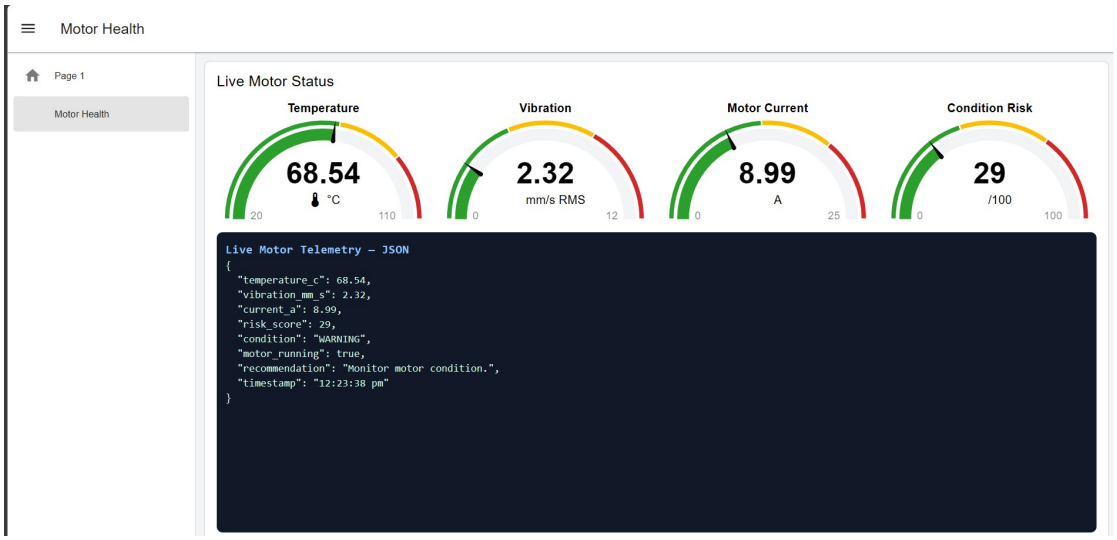


Figure 6.2 Node-RED Dashboard 2.0 motor health page (sample display)

Table 6.1 Dashboard gauge configuration

Gauge	Unit	Range	Colour bands (start values)
Temperature	°C	20 to 110	Green from 20, amber from 70, red from 90
Vibration	mm/s RMS	0 to 12	Green from 0, amber from 4.5, red from 8
Motor Current	A	0 to 25	Green from 0, amber from 12, red from 18
Condition Risk	/100	0 to 100	Green from 0, amber from 40, red from 70

The temperature, vibration and current bands match the firmware thresholds in Table 4.1. The risk gauge bands (40 and 70) are a visual guide only; the firmware itself classifies HIGH RISK at a score of 60 or more or when two warnings are active.

The screenshot in Figure 6.2 was captured with sample values to show the layout. Its risk score of 29 and its recommendation text cannot be produced by the firmware in Appendix 1, whose scores are built from the weights in Table 4.2, so the figure should be read as an illustration of the interface and not as an output of the firmware.

6.4 DATA PROCESSING FLOW

1. The ESP32 acquires the simulated sensor readings.
2. The ESP32 calculates the risk score and the motor condition.
3. The ESP32 publishes the telemetry through MQTT.
4. Node-RED receives the message from the telemetry topic.
5. The Parse telemetry function extracts and converts the fields.
6. The numeric values go to the corresponding gauges.
7. The complete object goes to the live JSON panel.

This arrangement keeps the embedded control and communication separate from the visualisation, so the dashboard can be changed without modifying the firmware.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 FUNCTIONAL BEHAVIOUR

The potentiometers allow the simulated temperature, vibration and current to be varied while the system runs. The ESP32 converts the inputs to engineering units and evaluates them against the thresholds. When a warning threshold is exceeded, the condition changes and an alarm message is published. When a trip threshold is exceeded, the green LED and contactor indicator switch off and the red LED and buzzer switch on.

Table 7.1 gives the condition and risk score that the firmware logic produces for typical combinations of warnings. The values follow from the rules in Tables 4.2 and 4.3 and are not measured data. The persistence bonus is excluded; when it applies, 10 points are added and the score is limited to 100.

Table 7.1 Condition and risk score produced by the firmware logic

Parameters above warning (no trip)	Risk score	Condition	Alarm sent
None	0	NORMAL	No
Temperature only	25	WARNING	Yes
Vibration only	40	WARNING	Yes
Current only	25	WARNING	Yes
Temperature and current	50	HIGH RISK	Yes
Temperature and vibration	65	HIGH RISK	Yes
Vibration and current	65	HIGH RISK	Yes
Temperature, vibration and current	90	HIGH RISK	Yes
Any parameter above its trip threshold	As above	CRITICAL	Yes

7.2 DISCUSSION

The prototype shows that sensing, embedded processing, MQTT communication, threshold-based fault detection and dashboard visualisation can be combined in one working chain. The two-second publishing interval gives the operator a near-real-time view, and the separate alarm topic makes abnormal events easy to pick out. The recommendation text gives the operator a first action for each condition.

Vibration carries the largest weight (40 points) in the risk score. This reflects the importance of vibration as an indicator of mechanical condition, and it also means that a vibration warning alone does not reach the HIGH RISK condition unless another warning is active.

7.3 VALIDITY OF THE RESULTS

The results show that the architecture and the monitoring logic work in a simulated environment. They do not establish diagnostic accuracy on a real induction motor, because no actual motor sensors, calibrated measurements or field validation were used. The thresholds are illustrative, and the risk score is a rule-based heuristic and not a trained machine-learning model.

CHAPTER 8

APPLICATIONS, ADVANTAGES AND LIMITATIONS

8.1 APPLICATIONS

The architecture can be adapted to several industrial and infrastructure settings:

- **Manufacturing plants:** conveyors, machine tools, pumps and production equipment.
- **Pumping stations:** observing temperature, vibration and current to identify abnormal operation.
- **Industrial automation:** bringing equipment status and fault indications into one monitoring interface.
- **Building services:** ventilation, cooling equipment and water circulation motors.
- **Smart infrastructure:** remote visibility of distributed electromechanical equipment.

Actual deployment needs suitable sensors, validated thresholds, secure communication and application-specific control and safety engineering.

8.2 ADVANTAGES

1. Continuous monitoring of the simulated motor parameters.
2. Early identification of abnormal operating conditions.
3. Lightweight telemetry and command communication through MQTT.
4. Central visualisation through a web-based dashboard.
5. Modular separation of acquisition, assessment, communication and visualisation.

6. Remote supervision and demonstration of control commands.
7. Testing without a physical motor or costly industrial sensors.
8. A base for later integration with real equipment and historical-data analysis.

8.3 LIMITATIONS

1. The sensor inputs are potentiometers and not calibrated sensors.
2. The risk score uses predefined rules and thresholds and not a trained predictive model.
3. Remaining useful life and the probability of failure are not estimated.
4. The system has not been validated under real industrial conditions.
5. The public MQTT broker is unencrypted and shared, so it is unsuitable for secure deployment.
6. The contactor is represented by an LED and no physical motor is switched.
7. The thresholds are illustrative and must be validated for the chosen motor and environment.
8. The dashboard depends on network connectivity and successful message processing.

CHAPTER 9

COURSE OUTCOME MAPPING

9.1 GENERAL

The project was carried out for the course Mechatronics and IoT. Table 9.1 maps the work to Course Outcomes CO3, CO4 and CO5, and the following sections explain how each outcome is addressed.

Table 9.1 Mapping of the project to the course outcomes

CO	Outcome	Where addressed
CO3	Analyse communication protocols and control strategies for IoT-based mechatronic systems	Chapters 4, 5 and 6
CO4	Develop a Mechatronics and IoT toolbox integrating sensors, actuators and communication systems	Chapters 3, 5 and 6
CO5	Collaborate to design a project addressing global challenges in manufacturing, healthcare or smart cities	Chapters 1, 7 and 8

9.2 CO3: COMMUNICATION PROTOCOLS AND CONTROL STRATEGIES

The project uses MQTT as a lightweight protocol for condition monitoring. The ESP32 publishes periodic telemetry and alarm messages and subscribes to a command topic. Wi-Fi provides the network connection and JSON gives a structured format for the measurements and motor status.

The control strategy is threshold based. Warnings produce maintenance recommendations, and critical conditions trigger the simulated trip outputs. The running state depends on the enable command and the trip condition. The work therefore illustrates how communication protocols,

embedded decision-making, remote supervision and actuator-state control are related in an IoT-based mechatronic system.

9.3 CO4: MECHATRONICS AND IoT TOOLBOX

The project integrates the ESP32, analog sensor simulation, LED and buzzer outputs, Wi-Fi, MQTT, JSON serialisation and a Node-RED Dashboard 2.0 interface. The analog input layer represents the sensing subsystem, the LEDs and buzzer represent the actuation and indication subsystem, MQTT links the embedded device to the monitoring application, and Node-RED processes and displays the data.

Together these tools form a modular prototype that can be extended with real temperature sensors, vibration sensors, current transducers, secured communication and industrial control hardware.

9.4 CO5: ADDRESSING INDUSTRIAL CHALLENGES

The project addresses the industrial problem of unexpected equipment failure and unplanned downtime. By showing continuous monitoring and early identification of abnormal conditions, it illustrates how IoT technology can support maintenance planning and equipment availability.

The modular architecture gives a base for collaborative development across mechanical engineering, electronics, embedded programming, communication systems and dashboard design, and it can be adapted to manufacturing and smart infrastructure. The present work is a simulation-based prototype. Its contribution is the demonstrated architecture and monitoring logic, not a proven reduction in downtime or maintenance cost.

CHAPTER 10

CONCLUSION AND FUTURE SCOPE

10.1 CONCLUSION

This project demonstrated an IoT-based condition monitoring and predictive maintenance prototype for an industrial induction motor using an ESP32, MQTT and Node-RED Dashboard 2.0. Simulated inputs represent motor temperature, vibration and current, and the ESP32 applies predefined thresholds to identify abnormal conditions.

The system publishes structured telemetry and alarm messages through MQTT, accepts start, stop and reset commands, and shows the running and trip states with LEDs and a buzzer. The Node-RED dashboard presents the readings on gauges and a live JSON panel.

The project shows the practical integration of sensing, embedded control, communication, data processing and visualisation, and it supports Course Outcomes CO3, CO4 and CO5. Although it is limited to simulation and a rule-based risk assessment, it provides a base for development with real sensors, secure communication, historical data analysis and validated predictive-maintenance algorithms.

10.2 FUTURE SCOPE

- **Real sensor integration:** replace the potentiometers with temperature sensors, vibration sensors and current transducers.
- **Historical data storage:** use a time-series database to store measurements and identify trends.
- **Machine-learning-based prediction:** train and validate a model on representative operating and failure data to classify faults and estimate degradation.

- **Secure MQTT communication:** use a private broker with TLS, authenticated clients and topic-level access control.
- **Advanced fault diagnosis:** distinguish bearing faults, misalignment, imbalance, overheating and abnormal electrical loading.
- **Improved dashboard:** add historical charts, configurable alerts, event logs and maintenance records.
- **Industrial integration:** connect the platform to a PLC or industrial gateway through a suitable industrial interface.
- **Safety engineering:** add independent protective devices, verified interlocks and controlled restart procedures before any real motor control.

APPENDIX 1

ESP32 FIRMWARE SOURCE CODE

Arduino (C++) firmware for the ESP32 as simulated in Wokwi, reproduced from the project source. The code is the basis of Chapters 4 and 5.

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>


// Industrial Motor Predictive Maintenance - Wokwi simulation
// Analog potentiometers simulate sensor outputs:
// GPIO34 = temperature, GPIO35 = vibration, GPIO32 = motor current.
// Public MQTT brokers are for demonstration only; use an authenticated broker for
real deployments.


const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_PASSWORD = "";
const char* MQTT_HOST = "broker.hivemq.com";
const uint16_t MQTT_PORT = 1883;


// Change this topic suffix to something unique before running multiple
demonstrations.
const char* TOPIC_TELEMETRY = "mechatronics/motorpm/demo2026/device01/telemetry";
const char* TOPIC_ALARM     = "mechatronics/motorpm/demo2026/device01/alarm";
const char* TOPIC_COMMAND   = "mechatronics/motorpm/demo2026/device01/command";


constexpr int PIN_TEMP = 34;
constexpr int PIN_VIB  = 35;
constexpr int PIN_CURR = 32;
constexpr int PIN_RUN_LED = 26;   // Green = motor running
constexpr int PIN_TRIP_LED = 27;  // Red = trip / severe fault
constexpr int PIN_RELAY   = 25;   // Simulated contactor output (LED indicator)
constexpr int PIN_BUZZER  = 14;


constexpr float TEMP_WARN_C = 70.0;
constexpr float TEMP_TRIP_C = 90.0;
constexpr float VIB_WARN_MM_S = 4.5;
constexpr float VIB_TRIP_MM_S = 8.0;
constexpr float CURRENT_WARN_A = 12.0;
constexpr float CURRENT_TRIP_A = 18.0;


WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);


bool motorEnabled = true;
bool manualTrip = false;
unsigned long lastPublish = 0;
unsigned long lastReconnectAttempt = 0;
unsigned long sampleCount = 0;
float previousTemp = 0;
float previousVib = 0;
float previousCurrent = 0;
int consecutiveWarnings = 0;


float readScaled(int pin, float minValue, float maxValue) {
  int raw = analogRead(pin);
  return minValue + (maxValue - minValue) * (float)raw / 4095.0f;
}
```

```
void setMotorOutputs(bool running, bool trip) {
  digitalWrite(PIN_RUN_LED, running ? HIGH : LOW);
  digitalWrite(PIN_RELAY, running ? HIGH : LOW);
  digitalWrite(PIN_TRIP_LED, trip ? HIGH : LOW);
  digitalWrite(PIN_BUZZER, trip ? HIGH : LOW);
}


void mqttCallback(char* topic, byte* payload, unsigned int length) {
  String message;
  for (unsigned int i = 0; i < length; i++) message += (char)payload[i];
  message.trim();


  if (String(topic) == TOPIC_COMMAND) {
    message.toLowerCase();
    if (message == "stop") {
      motorEnabled = false;
      Serial.println("Remote command: STOP");
    } else if (message == "start") {
      motorEnabled = true;
      manualTrip = false; // demo reset; real systems need a safe, verified reset
procedure
      Serial.println("Remote command: START / reset requested");
    } else if (message == "reset") {
      manualTrip = false;
      Serial.println("Remote command: RESET");
    }
  }
}


bool connectMqtt() {
  String clientId = "Wokwi-MotorPM-" + String((uint32_t)ESP.getEfuseMac(), HEX);
  if (mqtt.connect(clientId.c_str())) {
    mqtt.subscribe(TOPIC_COMMAND);
    Serial.println("MQTT connected; subscribed to command topic.");
    return true;
  }
  Serial.printf("MQTT connection failed, state=%d\n", mqtt.state());
  return false;
}


void setup() {
  Serial.begin(115200);
  analogReadResolution(12);


  pinMode(PIN_RUN_LED, OUTPUT);
  pinMode(PIN_TRIP_LED, OUTPUT);
  pinMode(PIN_RELAY, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);
  setMotorOutputs(false, false);


  Serial.println("\nIndustrial Motor Predictive Maintenance");
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD, 6);
  Serial.print("Connecting to Wokwi-GUEST");
  unsigned long start = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - start < 20000) {
    delay(250);
    Serial.print(".");
  }
  Serial.println();


  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setCallback(mqttCallback);
  mqtt.setBufferSize(512);


  if (WiFi.status() == WL_CONNECTED) {
    Serial.print("Wi-Fi connected, IP: ");
    Serial.println(WiFi.localIP());
    connectMqtt();
```

```
  } else {
    Serial.println("Wi-Fi not connected yet; will retry in loop.");
  }
}


void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD, 6);
    delay(100);
    return;
  }


  if (!mqtt.connected()) {
    if (millis() - lastReconnectAttempt > 3000) {
      lastReconnectAttempt = millis();
      connectMqtt();
    }
  } else {
    mqtt.loop();
  }


  if (millis() - lastPublish < 2000) {
    delay(10);
    return;
  }
  lastPublish = millis();
  sampleCount++;


  // Sensor values are deliberately simulated by potentiometer positions.
  float temperature = readScaled(PIN_TEMP, 25.0, 110.0);       // °C
  float vibration = readScaled(PIN_VIB, 0.0, 12.0);            // mm/s RMS
(simulated)
  float current = readScaled(PIN_CURR, 0.0, 25.0);             // A (simulated)


  bool tempWarn = temperature >= TEMP_WARN_C;
  bool vibWarn = vibration >= VIB_WARN_MM_S;
  bool currWarn = current >= CURRENT_WARN_A;


  bool tempTrip = temperature >= TEMP_TRIP_C;
  bool vibTrip = vibration >= VIB_TRIP_MM_S;
  bool currTrip = current >= CURRENT_TRIP_A;


  int warningCount = (tempWarn ? 1 : 0) + (vibWarn ? 1 : 0) + (currWarn ? 1 : 0);
  if (warningCount > 0) consecutiveWarnings++;
  else consecutiveWarnings = 0;


  // Transparent, explainable heuristic score; not a trained predictive model.
  int riskScore = (tempWarn ? 25 : 0) + (vibWarn ? 40 : 0) + (currWarn ? 25 : 0);
  if (consecutiveWarnings >= 3 && warningCount > 0) riskScore = min(100, riskScore +
10);


  String condition = "NORMAL";
  String recommendation = "Continue operation; maintain routine inspection.";
  if (tempTrip || vibTrip || currTrip || manualTrip) {
    condition = "CRITICAL";
    recommendation = "Motor tripped: inspect the indicated sensor/fault before
restart.";
  } else if (warningCount >= 2 || riskScore >= 60) {
    condition = "HIGH RISK";
    recommendation = "Schedule inspection soon; check bearing, cooling and load.";
  } else if (warningCount == 1) {
    condition = "WARNING";
    if (tempWarn) recommendation = "Check ventilation, cooling fan and bearing
temperature.";
    else if (vibWarn) recommendation = "Inspect bearing, alignment, balance and
mounting.";
    else recommendation = "Check mechanical load, supply and driven equipment.";
  }
```

```
  bool trip = tempTrip || vibTrip || currTrip || manualTrip;
  bool running = motorEnabled && !trip;
  setMotorOutputs(running, trip);


  StaticJsonDocument<512> doc;
  doc["device"] = "motor-01";
  doc["sample"] = sampleCount;
  doc["temperature_c"] = temperature;
  doc["vibration_mm_s"] = vibration;
  doc["current_a"] = current;
  doc["risk_score"] = riskScore;
  doc["condition"] = condition;
  doc["motor_running"] = running;
  doc["motor_enabled"] = motorEnabled;
  doc["trip"] = trip;
  doc["recommendation"] = recommendation;
  doc["uptime_s"] = millis() / 1000;


  char payload[512];
  size_t n = serializeJson(doc, payload, sizeof(payload));


  Serial.printf("T=%.1f C | Vib=%.2f mm/s | I=%.2f A | Risk=%d | %s | Motor=%s\n",
                temperature, vibration, current, riskScore, condition.c_str(),
                running ? "RUNNING" : "STOPPED");


  if (mqtt.connected()) {
    mqtt.publish(TOPIC_TELEMETRY, payload, false);
    if (trip || warningCount > 0) {
      StaticJsonDocument<256> alarm;
      alarm["device"] = "motor-01";
      alarm["condition"] = condition;
      alarm["temperature_c"] = temperature;
      alarm["vibration_mm_s"] = vibration;
      alarm["current_a"] = current;
      alarm["recommendation"] = recommendation;
      char alarmPayload[256];
      serializeJson(alarm, alarmPayload, sizeof(alarmPayload));
      mqtt.publish(TOPIC_ALARM, alarmPayload, false);
    }
  }


  previousTemp = temperature;
  previousVib = vibration;
  previousCurrent = current;
}
```

APPENDIX 2

NODE-RED PARSE TELEMETRY FUNCTION AND FLOW CONFIGURATION

The Parse telemetry function node of the Node-RED flow (reproduced with line breaks for readability) is listed below. It has five outputs, wired in order to the temperature, vibration, current and risk gauges and to the live telemetry JSON template.

```
let p = msg.payload;
if (typeof p === 'string') {
  try { p = JSON.parse(p); } catch(e) { return null; }
}
if (p && p.payload && typeof p.payload === 'object') p = p.payload;
if (!p || typeof p !== 'object') return null;
const n = (v) => Number.isFinite(Number(v)) ? Number(v) : 0;
const data = {
  temperature_c: n(p.temperature_c),
  vibration_mm_s: n(p.vibration_mm_s),
  current_a: n(p.current_a),
  risk_score: n(p.risk_score),
  condition: p.condition || 'NO DATA',
  motor_running: !!p.motor_running,
  recommendation: p.recommendation || 'No recommendation received',
  timestamp: new Date().toLocaleTimeString()
};
return [{payload:data.temperature_c}, {payload:data.vibration_mm_s},
        {payload:data.current_a}, {payload:data.risk_score},
{payload:data}];
```

The main node settings of the flow are given in Table A2.1.

Table A2.1 Main node settings of the Node-RED flow

Item	Setting
MQTT broker	broker.hivemq.com, port 1883, no TLS, MQTT protocol version 4
MQTT input topic	mechatronics/motorpm/demo2026/device01/telemetry, QoS 0
Parse telemetry	Function node, 5 outputs
Gauges	Four half-circle needle gauges (Temperature, Vibration, Motor Current, Condition Risk)
Live telemetry JSON	Dashboard 2.0 template node, 12 columns wide
Debug node	Attached to the temperature gauge output, prints the

Item	Setting
	payload to the sidebar
Dashboard page	Motor Health, path /motor-health, base path /dashboard

REFERENCES

1. Blanchon, B. (n.d.) ArduinoJson Documentation. Available at: https://arduinojson.org/
2. Espressif Systems (n.d.) ESP32 Series Documentation. Available at: https://docs.espressif.com/
3. FlowFuse (n.d.) Node-RED Dashboard 2.0 Documentation. Available at: https://dashboard.flowfuse.com/
4. Mobley, R.K. (2002) An Introduction to Predictive Maintenance, 2nd edn. Butterworth-Heinemann.
5. OASIS (2014) MQTT Version 3.1.1 Specification. Available at: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/
6. O'Leary, N. (n.d.) PubSubClient Arduino Library. Available at: https://github.com/knolleary/pubsubclient
7. OpenJS Foundation (n.d.) Node-RED Documentation. Available at: https://nodered.org/docs/
8. Wokwi (n.d.) ESP32 Simulation Documentation. Available at: https://docs.wokwi.com/